DOCKER MASTERCLASS

Storage & Memory — Pre-Read Material

Everything you need to know before the session

BY AKSHAT GUPTA (akshatawsac@gmail.com)

Topics Covered

1. Docker Prune & Dangling Images
2. Docker Volume Types
3. Named vs Unnamed Volumes
4. Docker Named Volume (Deep Dive)
5. Docker Bind Mounts
6. Netflix Case Study — When to Use Which
7. Container Memory Management

Chapter 1: Docker Prune & Dangling Images

1.1 What Problem Does Docker Prune Solve?

When you work with Docker day-to-day, your system quietly accumulates a lot of dead weight — stopped containers that were never cleaned up, old images from builds you ran last week, cache from every `docker build` command, and unused networks. None of these are doing any useful work, but they sit on disk eating space.

Over time this adds up fast, especially on a CI/CD server that builds dozens of images a day. Disk fills up, builds start failing, and someone has to scramble to clean things up manually.

> 🎬 **Netflix Real-World Story**
>
> Netflix runs thousands of CI/CD pipelines every single day. Each pipeline creates a Docker image.
>
> After 48 hours of builds with no cleanup, servers were losing 1 TB of disk space.
>
> Engineers started getting build failures and disk-full errors at 3 AM.
>
> The fix? A `docker system prune -a` command on a scheduled cron job. Saved them on-call nightmares.

1.2 What Is a Dangling Image?

A dangling image is a Docker image that has no tag and no container currently using it. Think of it as an orphan — created but abandoned.

Here is how they get created: Suppose you build your app image today with the tag `myapp:latest`. Tomorrow, you change the code and build again with the same tag. Docker cannot delete the old image immediately (something might still be referencing it), so it removes the tag but leaves the image on disk. That old, untagged image is now "dangling".

```
# See all dangling images
docker images -f "dangling=true"

# Output looks like:
REPOSITORY   TAG       IMAGE ID       CREATED        SIZE
<none>       <none>    a1b2c3d4e5f6   2 days ago     312MB
<none>       <none>    9f8e7d6c5b4a   5 days ago     289MB
```

1.3 The Prune Command Family

Docker provides a set of prune commands that clean up different things:

Command	What It Removes	When to Use
docker image prune	Only dangling images (no tag, no container)	Light cleanup — keeps your named images
docker container prune	All stopped containers	After testing many containers
docker volume prune	Volumes not attached to any container	Careful — this deletes data!
docker network prune	Networks not used by any container	Routine network cleanup
docker system prune	Containers + networks + dangling images + build cache	Regular cleanup pass
docker system prune -a	Everything above + ALL unused images (not just dangling)	Deep clean on CI servers

⚠️ **Warning**

docker volume prune will delete volumes not attached to a running container.
If your database volume is detached (container stopped), it WILL be deleted.
Always run docker volume ls and verify before pruning volumes.

1.4 Practical Commands

```
# Remove only dangling images
docker image prune

# Remove ALL unused images (not just dangling)
docker image prune -a

# Full system cleanup (most common)
docker system prune -a

# Schedule cleanup via cron (run at 2 AM daily)
0 2 * * * /usr/bin/docker system prune -a -f
```

1.5 Key Takeaways

- Dangling images = images with no tag and no running container. They pile up silently.
- docker system prune -a is your best friend for deep cleaning.
- Always exclude volumes from prune unless you know what you're doing.
- Schedule prune on CI/CD servers. Don't wait for disk-full alerts.

Chapter 2: Docker Volume Types

2.1 Why Do We Even Need Volumes?

By default, Docker containers are completely isolated and stateless. Any data written inside a container's filesystem is tied to that container's lifecycle. When the container stops, restarts, or is deleted — that data is gone.

This is fine for stateless apps like a web server serving static pages. But the moment you need persistent data — a database, user uploads, application logs, configuration files — you need a volume.

Simple rule: If you can afford to lose the data when the container dies, skip volumes. If you cannot lose it, use a volume.

2.2 The Three Types of Docker Storage

Docker gives you three fundamentally different ways to give a container access to storage:

Type 1: Named Volumes

Docker creates and manages a dedicated storage area on the host, and you give it a name you choose. The data lives at /var/lib/docker/volumes/<name>/_data on the host, but Docker abstracts all of that away from you. You just refer to it by name.

```
docker run -v mydata:/app/data myimage
# 'mydata' is the volume name you chose
# '/app/data' is where it appears inside the container
```

Type 2: Anonymous (Unnamed) Volumes

Identical to named volumes in how they work, but Docker picks a random UUID as the name instead of you choosing one. They are created when you specify a container-side path without a name prefix.

```
docker run -v /app/data myimage
# No name prefix = Docker picks a UUID name
# Example: -v a3f8b2c1...:/app/data
```

Type 3: Bind Mounts

You directly link a specific path on your host machine to a path inside the container. There is no Docker-managed abstraction — the container directly reads and writes the host directory.

```
docker run -v /home/user/myproject:/app myimage
# /home/user/myproject = actual path on your host
# /app = where it appears inside the container
```

2.3 Side-by-Side Comparison

Feature	Named Volume	Bind Mount
Who controls storage location?	Docker	You (the host path)
Portability	High — works on any Docker host	Low — tied to a specific host path
Performance	Optimized by Docker	Slower on Mac/Windows (I/O overhead)
Use in production?	Yes — recommended	Avoid for data; OK for configs
Use in development?	Yes	Yes — great for live code reload
Data survives container removal?	Yes	Yes (host file stays)
Easy to back up?	Yes — docker volume commands	Yes — it's just a host directory

Chapter 3: Named vs Unnamed Volumes

3.1 The Critical Difference

Both named and unnamed volumes are managed by Docker and work the same way internally. The difference is entirely in how you reference them — and that one difference has enormous consequences for data safety.

> 😭 **Data Loss Incident**
>
> A developer at a growing startup ran: docker run -v /app/uploads myapp
> The container stored user-uploaded profile photos in /app/uploads.
> The container was restarted after a config change. Docker saw no name and could not link the new container to the old volume. The old volume became an orphan.
> When someone later ran docker volume prune — 200 GB of user photos were deleted.
> No recovery was possible. The team had no backup.

3.2 Named Volumes — The Right Way

A named volume is created with a name you define. That name is the permanent identifier. Regardless of how many times you stop, start, recreate, or update the container — as long as you use the same volume name, the same data is there.

```
# Create the volume explicitly (or Docker creates it on first run)
docker volume create user_uploads

# Always use the name
docker run -v user_uploads:/app/uploads myapp

# Even after removing and recreating the container:
docker rm myapp-container
docker run -v user_uploads:/app/uploads myapp  # Data is still there
```

3.3 Unnamed Volumes — When They're Acceptable

Unnamed volumes are not inherently bad — they just should not be used for data you care about. Use cases where they are fine:

- Build stages in multi-stage Dockerfiles where intermediate data is discarded
- Temporary scratch space inside a container during a job
- Data explicitly meant to be ephemeral and discarded when the container dies

> ✅ **Rule of Thumb**
> If you are asking 'will I miss this data if the container is removed?' — use a named volume.
> If the answer is 'no, it's temporary' — an unnamed volume is acceptable.
> In production systems: always use named volumes. No exceptions.

3.4 How to Find Orphaned Volumes

```
# List all volumes
docker volume ls

# See volumes NOT attached to any container
docker volume ls -f dangling=true

# Inspect a specific volume
docker volume inspect user_uploads
```

Chapter 4: Docker Volumes — Deep Dive

4.1 How Docker Volumes Work Internally

When you create a named volume, Docker stores it at a specific location on the host filesystem:

```
/var/lib/docker/volumes/<volume-name>/_data
```

You rarely need to touch this path directly — Docker provides commands to manage it. But knowing it exists helps you understand backups and disaster recovery.

The lifecycle of a named volume looks like this:

1. You create the volume (or it is auto-created on first docker run)
2. You attach it to a container at a specific mount path
3. The container reads and writes data as if it were a normal directory
4. When the container stops or is removed, the volume data stays untouched
5. A new container with the same volume name picks up the exact same data

4.2 Volume Commands Cheatsheet

Command	What It Does
`docker volume create mydata`	Creates a named volume called mydata
`docker volume ls`	Lists all volumes on this Docker host
`docker volume inspect mydata`	Shows full details: mount path, creation time, etc.
`docker volume rm mydata`	Deletes the volume and all data inside it (irreversible)
`docker volume prune`	Deletes all volumes not attached to any container
`docker run -v mydata:/app/data img`	Attaches mydata volume to /app/data inside the container

4.3 Volumes in Docker Compose

In a Docker Compose setup, the recommended pattern is to declare volumes at the bottom of the file and reference them in each service that needs them. This ensures all services share the exact same volume, and the volume persists across `docker compose down` and `docker compose up` cycles.

```
version: '3.8'
services:
```

```
  web:
    image: myapp:latest
    volumes:
      - app_data:/app/uploads    # Reference the named volume

  db:
    image: postgres:15
    volumes:
      - pg_data:/var/lib/postgresql/data

volumes:
  app_data:      # Docker creates and manages this
  pg_data:       # Docker creates and manages this
```

> 💡 **Pro Tip**
> docker compose down does NOT delete named volumes by default.
> To also delete volumes: docker compose down -v
> This is intentional — Docker protects your data from accidental deletion.

4.4 Backing Up and Restoring Volumes

Since volumes are just directories on disk, backing them up is straightforward:

```
# BACKUP: Create a tar archive of volume contents
docker run --rm -v mydata:/data -v $(pwd):/backup ubuntu \
  tar czf /backup/mydata-backup.tar.gz -C /data .

# RESTORE: Extract backup into a volume
docker run --rm -v mydata:/data -v $(pwd):/backup ubuntu \
  tar xzf /backup/mydata-backup.tar.gz -C /data
```

Chapter 5: Docker Bind Mounts

5.1 What Is a Bind Mount?

A bind mount is a direct link between a specific directory on your host machine and a directory inside the container. Unlike named volumes (where Docker picks the storage location), with a bind mount you are in full control of where the data lives.

The container and the host share the same files in real time. If you edit a file on the host, the container sees the change immediately. If the container writes a file, you can see it on the host immediately.

```
# Syntax
docker run -v <host-path>:<container-path> image

# Real examples
docker run -v /home/alice/project:/app myapp          # Full path
docker run -v $(pwd):/app myapp                       # Current directory
docker run -v $(pwd)/config:/etc/nginx:ro myapp       # Read-only (:ro)
```

🎬 **Netflix Dev Story**

A Netflix backend developer was iterating on a Node.js microservice.
Her workflow: edit code -> docker build -> docker run -> test -> repeat.
Each cycle took 45 seconds. She made ~200 changes per day = 2.5 hours wasted just waiting.
Solution: Bind mount the source code directory into the container + nodemon for auto-restart.
Edit code on host -> container sees it instantly -> nodemon restarts -> test immediately.
Saved her 2+ hours every single day.

5.2 The Developer Workflow

```
# Node.js example with live reload
docker run \
  -v $(pwd):/app \
  -w /app \
  -p 3000:3000 \
  node:20 \
  npx nodemon server.js

# Python Flask with live reload
docker run \
  -v $(pwd):/app \
```

```
  -e FLASK_ENV=development \
  -p 5000:5000 \
  python:3.11 python app.py
```

5.3 Read-Only Bind Mounts

Adding :ro at the end of the volume flag makes the mount read-only from the container's perspective. The container can read the files but cannot modify them. This is a security best practice for config files and secrets.

```
# Mount nginx config read-only
docker run -v /etc/myapp/nginx.conf:/etc/nginx/nginx.conf:ro nginx

# Mount SSL certificates read-only
docker run -v /etc/ssl/certs:/etc/ssl/certs:ro myapp
```

5.4 Bind Mounts — Limitations to Know

- Path must exist on the host. Docker will not create it automatically (unlike volumes).
- Portability suffers. The container definition references a specific host path. Move to another machine and the path must exist there too.
- Performance on Mac and Windows is slower than native Linux or named volumes, due to the virtualization layer in Docker Desktop.
- Security risk: a container with a bind mount to a sensitive host directory (e.g. /) could modify system files. Always use :ro when the container only needs to read.

5.5 Named Volume vs Bind Mount — Decision Guide

Situation	Use Named Volume	Use Bind Mount
Production database data	YES	No
User-uploaded files in prod	YES	No
Local development code (live reload)	No	YES
Injecting nginx.conf into container	No	YES (read-only)
Sharing config secrets from host	No	YES (read-only)
Persistent app state across restarts	YES	Works but not recommended

CI/CD build artifacts	YES	Acceptable

Chapter 6: Netflix Case Study — When to Use Which

Netflix operates one of the most complex containerised environments in the world — thousands of microservices, continuous deployments, and services ranging from databases to real-time video processing. Each service has a very different storage need. Here is how their teams would approach volume choice:

6.1 The Database Team — Named Volume

Scenario

The recommendations database stores watch history for 260 million users.
It runs on PostgreSQL inside a Docker container.
Containers are restarted during deployments, patches, and scaling events.

Choice: Named Volume

Reason: The data must survive every container restart and redeployment. Docker manages the storage safely. Using a named volume means the data is there regardless of how many times the container is recreated.

```
docker run \
  -v postgres_data:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret \
  postgres:15
```

6.2 The Frontend Developer — Bind Mount

Scenario

A developer is building the Netflix web UI — a React application.
She needs to see her changes reflected instantly without rebuilding images.
She works locally on her MacBook, running the dev server inside Docker.

Choice: Bind Mount

Reason: Her source code lives on her laptop. A bind mount lets the container see her files directly. Changes she makes in her editor instantly appear inside the container — no rebuild needed.

```
docker run \
  -v $(pwd)/src:/app/src \
  -p 3000:3000 \
  node:20 npm start
```

6.3 The Streaming Team — Anonymous Volume

Scenario

The video transcoding service converts video files into multiple resolutions.
Each transcoding job creates a large temporary buffer during processing.
Once the job is done, the buffer files are useless and should be discarded.

Choice: Anonymous Volume (or just container filesystem)

Reason: The data is completely ephemeral. It only exists for the duration of one transcoding job. There is no reason to name or preserve it. When the container exits, the data should disappear automatically.

```
docker run \
  -v /tmp/transcode \
  netflix-transcoder:latest
```

6.4 The Platform Team — Bind Mount (Read-Only)

Scenario

The API gateway needs a specific nginx.conf and SSL certificates.
These files are managed by the platform team on the host server.
The nginx container must use them but should never be able to modify them.

Choice: Bind Mount (Read-Only)

Reason: The config files already exist on the host, managed by the infrastructure team. A bind mount gives the container access without duplicating the files. The :ro flag ensures the container cannot accidentally overwrite the configs.

```
docker run \
  -v /etc/netflix/nginx.conf:/etc/nginx/nginx.conf:ro \
  -v /etc/ssl/netflix:/etc/ssl/certs:ro \
  -p 443:443 \
  nginx:latest
```

6.5 Quick Decision Matrix

Your Situation	Volume Choice	Key Reason
Production database	Named Volume	Must survive restarts. Docker manages safely.

User file uploads	Named Volume	Cannot lose user data ever.
Application logs (prod)	Named Volume or Bind	Depends on if ops team manages log path.
Local dev code (live reload)	Bind Mount	Code lives on host. Instant feedback.
Injecting config files	Bind Mount (:ro)	Config managed outside Docker on host.
Temp processing buffers	Anonymous Volume	Ephemeral. Gone when container exits.
Build cache	Named Volume or Prune	Persist for speed; clean up regularly.

Chapter 7: Docker Container Memory Management

7.1 Why Memory Limits Matter

By default, a Docker container has no memory limit. It can consume as much RAM as the host has available. On a shared server running many containers, one runaway container — perhaps due to a memory leak or a traffic spike — can consume all available RAM, causing every other container on the host to crash or be killed by the operating system.

> 💀 **Netflix Production Incident**
>
> A recommendation engine container developed a memory leak in a third-party library.
> With no memory limit set, it steadily consumed RAM over 6 hours.
> It reached all 64 GB of server RAM. The OS killed other processes to survive.
> The payments service container was killed. Checkout was down for 14 minutes.
> Estimated revenue loss: $2 million. Root cause: no memory limit on a single container.
> Fix: Set -m 4g on the recommendation service. Reserve 2g. Memory-swap equal to limit.

7.2 The Three Memory Flags

Flag 1: --memory (or -m) — The Hard Limit

This is the maximum amount of memory a container is allowed to use. If the container tries to allocate memory beyond this limit, the Linux OOM (Out of Memory) killer will terminate the container process. Think of this as a hard ceiling.

```
docker run -m 512m myapp        # Limit to 512 megabytes
docker run -m 2g myapp          # Limit to 2 gigabytes
docker run --memory=1g myapp    # Full flag name
```

Analogy: Like a strict credit card limit. When you hit it, the transaction is declined — or in Docker's case, the process is killed.

Flag 2: --memory-reservation — The Soft Limit

This sets a soft guarantee. Docker will try to ensure the container has at least this much memory available, especially when the host is under memory pressure. Unlike the hard limit, exceeding the reservation does not kill the container — it just signals to Docker's scheduler that memory is needed.

```
docker run --memory-reservation 256m myapp
```

```
# Combined with hard limit (recommended pattern)
docker run -m 512m --memory-reservation 256m myapp
# Container guaranteed 256m, capped at 512m
```

Analogy: Like a reserved parking spot — it's yours when you need it, but on a quiet Sunday you might park somewhere else temporarily.

Flag 3: --memory-swap — The Swap Limit

Swap memory is slower disk-based memory that the OS uses when RAM is full. The --memory-swap flag controls the total amount of memory + swap the container can use.

--memory-swap value	Effect
Not set (default)	Container can use 2x its memory limit in swap (e.g. -m 512m = 1GB total)
Same as -m (e.g. both 512m)	Swap is disabled. Container crashes before using swap.
-1	Unlimited swap. Container can use as much swap as the host has.
Greater than -m	Specific swap amount. e.g. -m 512m --memory-swap 1g = 512m RAM + 512m swap

```
# Disable swap (fail fast — recommended for most prod services)
docker run -m 512m --memory-swap 512m myapp

# Allow some swap as overflow buffer
docker run -m 512m --memory-swap 1g myapp
```

Analogy: Swap is like overflow parking — slower, further away, but it prevents a full crash.

7.3 The Golden Rule for Production

✅ **Production Memory Best Practice**

Always set BOTH -m (hard limit) AND --memory-reservation (soft limit).

Reservation = 50% of the hard limit is a good starting point.

Example for a typical microservice:
docker run -m 1g --memory-reservation 512m --memory-swap 1g myapp

This means:
- Container is guaranteed 512m
- Container cannot exceed 1g of RAM
- Swap is disabled (fail fast — alerts fire before silent degradation)

7.4 Monitoring Memory Usage

```
# Live stats for all running containers
docker stats

# Stats for a specific container
docker stats my-container

# One-shot (no live update)
docker stats --no-stream

# Output includes: MEM USAGE / LIMIT, MEM %
CONTAINER ID   NAME         MEM USAGE / LIMIT   MEM %
a1b2c3d4e5f6   my-api       234MiB / 512MiB     45.7%
```

7.5 Setting Memory in Docker Compose

```
version: '3.8'
services:
  api:
    image: myapi:latest
    deploy:
      resources:
        limits:
          memory: 512m        # Hard limit (-m equivalent)
        reservations:
          memory: 256m        # Soft limit (--memory-reservation equivalent)
```

7.6 Key Takeaways

- No memory limit = one bad container can crash an entire server.
- -m sets the hard ceiling. OOM killer fires when it's breached.
- --memory-reservation sets the soft guarantee for scheduler hints.
- --memory-swap controls total RAM + swap. Set equal to -m to disable swap.
- Run docker stats regularly to understand real-world usage before setting limits.
- In Docker Compose, use the deploy.resources block for memory settings.

Quick Reference — All Commands

Prune Commands

```
docker image prune              # Remove dangling images
docker images -f "dangling=true" # List dangling images
docker container prune          # Remove stopped containers
docker volume prune             # Remove unused volumes (careful!)
docker system prune -a          # Full cleanup
```

Volume Commands

```
docker volume create mydata     # Create named volume
docker volume ls                # List all volumes
docker volume inspect mydata    # Inspect a volume
docker volume rm mydata         # Delete a volume
docker volume prune             # Delete unused volumes
```

Run With Storage

```
# Named volume
docker run -v mydata:/app/data image

# Bind mount (host path : container path)
docker run -v /host/path:/app image

# Bind mount current directory
docker run -v $(pwd):/app image

# Read-only bind mount
docker run -v /host/config:/etc/config:ro image

# Anonymous volume
docker run -v /app/tmp image
```

Memory Flags

```
docker run -m 512m image                          # Hard memory limit
docker run --memory-reservation 256m image        # Soft memory limit
docker run -m 512m --memory-swap 512m image       # Disable swap
```

```
docker run -m 1g --memory-reservation 512m image   # Recommended combo
docker stats                                        # Live memory stats
docker stats --no-stream                            # One-shot memory stats
```

End of Pre-Read Material — DevOps Docker Masterclass